

SQL SELF JOINS

Senior Developer Mastery Guide

All Patterns · Deep Theory · 30 Graded Exercises

Advanced Level — No Pattern Left Behind

TABLE OF CONTENTS

Chapter 1	Foundations & Mental Model
Chapter 2	Pattern 1 — Hierarchy & Adjacency List
Chapter 3	Pattern 2 — Consecutive & Lag/Lead Rows
Chapter 4	Pattern 3 — Within-Group Comparisons
Chapter 5	Pattern 4 — Graph Traversal (Recursive CTE + Self Join)
Chapter 6	Pattern 5 — De-duplication & Overlap Detection
Chapter 7	Pattern 6 — Transitive Closure
Chapter 8	Pattern 7 — Time-Series & Gaps
Chapter 9	Pattern 8 — Ranking Without Window Functions
Chapter 10	Performance, Anti-Patterns & Optimisation
Chapter 11	30 Graded Exercises
Chapter 12	Full Answer Key

Chapter 1 · Foundations & Mental Model

What is a Self Join?

A self join joins a table to itself by treating the same physical table as two logically distinct tables using different aliases. There is no special SQL keyword — it uses the same INNER / LEFT / CROSS JOIN syntax. What makes it conceptually unique is that both sides of the join predicate reference the same underlying storage.

◆ Canonical template

```
SELECT a.col, b.col

FROM my_table AS a

JOIN my_table AS b -- same table, different alias

ON a.some_key = b.other_key;
```

The Cartesian-Product Mental Model

Before any ON predicate, a self join produces N × N rows (Cartesian product). Your WHERE / ON clause selects the subset of that grid that satisfies your business rule. Always visualise the full grid first, then filter.

a.id	a.name	b.id	b.name	Pair type
1	Alice	1	Alice	Same row (often excluded)
1	Alice	2	Bob	Valid pair
1	Alice	3	Carol	Valid pair
2	Bob	1	Alice	Mirror of row 2 (often excluded)
2	Bob	2	Bob	Same row
2	Bob	3	Carol	Valid pair
3	Carol	1	Alice	Mirror of row 6
3	Carol	2	Bob	Mirror of row 3
3	Carol	3	Carol	Same row

Three Canonical Filters

- a.id <> b.id — Exclude same-row pairs — used in comparisons
- a.id < b.id — Exclude duplicates AND same-row — ordered unique pairs
- a.parent_id = b.id — Hierarchical traversal — parent-child relationship

Join Types in Self Joins

INNER JOIN — Only rows where both sides satisfy the predicate. Excludes NULLs.

LEFT JOIN — Keeps the 'a' side even if no matching 'b' row exists. Essential for finding roots/leaves in trees, or rows with no match.

CROSS JOIN — Generates every combination. Use intentionally for combinatorics.

■ *Golden rule: always alias both copies meaningfully (e.g., child/parent, curr/prev, a/b) — self-join bugs almost always come from confused aliases.*

Chapter 2 · Pattern 1 — Hierarchy & Adjacency List

The Schema

◆ **employees table** — used throughout this guide

```
CREATE TABLE employees (  
  
  emp_id INT PRIMARY KEY,  
  
  name VARCHAR(100),  
  
  dept VARCHAR(50),  
  
  salary DECIMAL(10,2),  
  
  manager_id INT REFERENCES employees(emp_id),  
  
  hire_date DATE  
  
);
```

emp_id	name	dept	salary	manager_id	hire_date
1	Sarah Chen	Engineering	180000	NULL	2018-01-15
2	James Park	Engineering	140000	1	2019-03-22
3	Maria Lopez	Engineering	130000	1	2020-06-10
4	David Kim	Marketing	120000	NULL	2017-09-01
5	Emma Wilson	Marketing	110000	4	2021-02-14
6	Liam Brown	Engineering	95000	2	2022-07-30
7	Aisha Osei	HR	105000	NULL	2016-04-05

2.1 Employee → Direct Manager

```
SELECT e.name AS employee,  
  
  m.name AS manager,  
  
  e.salary AS emp_salary,  
  
  m.salary AS mgr_salary  
  
FROM employees AS e
```

```
LEFT JOIN employees AS m -- LEFT keeps top-level (no manager)

ON e.manager_id = m.emp_id;
```

2.2 Employees Earning More Than Their Manager

```
SELECT e.name AS employee,

e.salary AS emp_salary,

m.name AS manager,

m.salary AS mgr_salary,

e.salary - m.salary AS overpay

FROM employees AS e

JOIN employees AS m ON e.manager_id = m.emp_id

WHERE e.salary > m.salary;
```

2.3 Multi-Level Path (3 hops)

```
-- Grandchild → Child → Parent in a single query

SELECT gc.name AS grandchild,

c.name AS child,

p.name AS grandparent

FROM employees AS gc

JOIN employees AS c ON gc.manager_id = c.emp_id

JOIN employees AS p ON c.manager_id = p.emp_id;
```

■ For unlimited depth use a Recursive CTE (Chapter 5). Multi-hop self joins are $O(N^k)$ — limit to known fixed depths only.

2.4 Finding Roots (no manager) and Leaves (no reports)

```
-- Roots: employees with no manager

SELECT e.name

FROM employees AS e
```

```
LEFT JOIN employees AS m ON e.manager_id = m.emp_id

WHERE e.manager_id IS NULL;

-- Leaves: employees who manage nobody

SELECT e.name

FROM employees AS e

LEFT JOIN employees AS sub ON sub.manager_id = e.emp_id

WHERE sub.emp_id IS NULL;
```

Chapter 3 · Pattern 2 — Consecutive & Lag/Lead Rows

Before window functions existed, self joins were the only way to access the previous or next row. Even today they offer flexibility window functions lack, such as joining on non-adjacent or conditional rows.

Schema — sales

```
CREATE TABLE sales (  
  
  sale_id INT PRIMARY KEY,  
  
  emp_id INT,  
  
  amount DECIMAL(10,2),  
  
  sale_date DATE  
  
);
```

3.1 Classic Lag: Previous Row Per Employee

```
-- For every sale, show the immediately preceding sale by the same employee  
  
SELECT curr.emp_id,  
  
  curr.sale_date AS current_date,  
  
  curr.amount AS current_amount,  
  
  prev.sale_date AS prev_date,  
  
  prev.amount AS prev_amount,  
  
  curr.amount - prev.amount AS delta  
  
FROM sales AS curr  
  
LEFT JOIN sales AS prev  
  
  ON prev.emp_id = curr.emp_id  
  
  AND prev.sale_date = (  
  
    SELECT MAX(p2.sale_date)
```

```

FROM sales AS p2

WHERE p2.emp_id = curr.emp_id

AND p2.sale_date < curr.sale_date

);

```

3.2 Running Total Using Self Join

```

-- Cumulative amount for each employee up to and including the current row

SELECT a.sale_id,

a.emp_id,

a.sale_date,

a.amount,

SUM(b.amount) AS running_total

FROM sales AS a

JOIN sales AS b

ON b.emp_id = a.emp_id

AND b.sale_date <= a.sale_date

GROUP BY a.sale_id, a.emp_id, a.sale_date, a.amount

ORDER BY a.emp_id, a.sale_date;

```

■ Running totals via self join are $O(N^2)$. Use window `SUM() OVER ()` in production for large datasets.

3.3 Detect Consecutive Increases

```

-- Rows where the amount rose compared to the prior row (same employee)

SELECT curr.emp_id,

curr.sale_date,

curr.amount,

prev.amount AS prev_amount

```

```
FROM sales AS curr

JOIN sales AS prev

ON prev.emp_id = curr.emp_id

AND prev.sale_date < curr.sale_date

AND NOT EXISTS ( -- gap-free: no row between prev and curr

SELECT 1 FROM sales AS mid

WHERE mid.emp_id = curr.emp_id

AND mid.sale_date > prev.sale_date

AND mid.sale_date < curr.sale_date

)

WHERE curr.amount > prev.amount;
```

Chapter 4 · Pattern 3 — Within-Group Comparisons

Compare every row to every other row within the same logical group (department, category, cohort). The `a.id < b.id` trick is key.

4.1 All Salary Pairs Within a Department

```
SELECT a.name AS emp1, a.salary AS sal1,

       b.name AS emp2, b.salary AS sal2,

       ABS(a.salary - b.salary) AS diff

FROM   employees AS a

JOIN   employees AS b

ON     a.dept = b.dept

AND    a.emp_id < b.emp_id -- unique unordered pairs only

ORDER BY a.dept, diff DESC;
```

4.2 Maximum Salary Gap Per Department

```
SELECT a.dept,

       MAX(a.salary - b.salary) AS max_gap,

       MAX(a.name) AS top_earner,

       MIN(b.name) AS low_earner

FROM   employees AS a

JOIN   employees AS b

ON     a.dept = b.dept

AND    a.salary > b.salary

GROUP BY a.dept;
```

4.3 N-th Highest Salary (Classic Interview Pattern)

```
-- 2nd highest salary in each department
```

```

SELECT dept, salary

FROM employees AS a

WHERE 1 = ( -- exactly 1 row has a higher salary

SELECT COUNT(DISTINCT b.salary)

FROM employees AS b

WHERE b.dept = a.dept

AND b.salary > a.salary

)

ORDER BY dept;

-- Generalised: N-th highest (set @n = desired rank)

SELECT dept, salary

FROM employees AS a

WHERE @n - 1 = (

SELECT COUNT(DISTINCT b.salary)

FROM employees AS b

WHERE b.dept = a.dept

AND b.salary > a.salary

)

ORDER BY dept;

```

4.4 Employees Hired After Every Member of Their Department

```

-- The last hire in each department

SELECT a.name, a.dept, a.hire_date

FROM employees AS a

WHERE NOT EXISTS (

```

```
SELECT 1

FROM employees AS b

WHERE b.dept = a.dept

AND b.hire_date > a.hire_date

);
```

Chapter 5 · Pattern 4 — Graph Traversal (Recursive CTE + Self Join)

Recursive CTEs traverse a graph of arbitrary depth. A self join is embedded inside the recursive member to extend the path one hop at a time.

5.1 Full Org-Chart Path (All Depths)

```
WITH RECURSIVE org_tree AS (  
  
    -- Anchor: root nodes  
  
    SELECT emp_id, name, manager_id,  
  
    CAST(name AS VARCHAR(1000)) AS path,  
  
    0 AS depth  
  
    FROM employees  
  
    WHERE manager_id IS NULL  
  
    UNION ALL  
  
    -- Recursive member: self join extends one level  
  
    SELECT e.emp_id, e.name, e.manager_id,  
  
    CONCAT(ot.path, ' → ', e.name),  
  
    ot.depth + 1  
  
    FROM employees AS e  
  
    JOIN org_tree AS ot ON e.manager_id = ot.emp_id  
  
    )  
  
    SELECT path, depth  
  
    FROM org_tree  
  
    ORDER BY path;
```

5.2 All Ancestors of a Given Employee

```
WITH RECURSIVE ancestors AS (  
  
  SELECT emp_id, name, manager_id, 0 AS level  
  
  FROM employees  
  
  WHERE name = 'Liam Brown' -- start node  
  
  UNION ALL  
  
  SELECT p.emp_id, p.name, p.manager_id, a.level + 1  
  
  FROM employees AS p  
  
  JOIN ancestors AS a ON a.manager_id = p.emp_id  
  
)  
  
SELECT name, level FROM ancestors ORDER BY level;
```

5.3 Cycle Detection (Safety Guard)

```
WITH RECURSIVE safe_tree AS (  
  
  SELECT emp_id, name, manager_id,  
  
  ARRAY[emp_id] AS visited,  
  
  false AS cycle  
  
  FROM employees WHERE manager_id IS NULL  
  
  UNION ALL  
  
  SELECT e.emp_id, e.name, e.manager_id,  
  
  st.visited || e.emp_id,  
  
  e.emp_id = ANY(st.visited) -- cycle flag  
  
  FROM employees AS e  
  
  JOIN safe_tree AS st ON e.manager_id = st.emp_id
```

```
WHERE NOT st.cycle
```

```
)
```

```
SELECT * FROM safe_tree WHERE cycle; -- show cyclic nodes
```

■ PostgreSQL supports `ARRAY[]`. In MySQL 8+ / SQL Server use a path string and `LOCATE/CHARINDEX` for cycle detection.

Chapter 6 · Pattern 5 — De-duplication & Overlap Detection

Schema — bookings

```
CREATE TABLE bookings (  
  
  booking_id INT PRIMARY KEY,  
  
  resource VARCHAR(50),  
  
  start_time TIMESTAMP,  
  
  end_time TIMESTAMP  
  
);
```

6.1 Find Overlapping Bookings for the Same Resource

```
SELECT a.booking_id AS booking1,  
  
  b.booking_id AS booking2,  
  
  a.resource,  
  
  GREATEST(a.start_time, b.start_time) AS overlap_start,  
  
  LEAST (a.end_time, b.end_time) AS overlap_end  
  
FROM bookings AS a  
  
JOIN bookings AS b  
  
ON a.resource = b.resource  
  
AND a.booking_id < b.booking_id -- unique pairs  
  
AND a.start_time < b.end_time -- overlap condition (Allen's interval algebra)  
  
AND a.end_time > b.start_time;
```

■ The two-predicate overlap check ($start1 < end2$ AND $end1 > start2$) is Allen's Interval Algebra 'overlap' relation — memorise it.

6.2 Delete True Duplicate Rows (keep lowest id)

```

-- Identify duplicates

SELECT a.booking_id AS keep, b.booking_id AS drop_this

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.start_time = b.start_time

AND a.end_time = b.end_time

AND a.booking_id < b.booking_id;

-- Delete in PostgreSQL / MySQL 8+

DELETE FROM bookings

WHERE booking_id IN (

SELECT b.booking_id

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.start_time = b.start_time

AND a.end_time = b.end_time

AND a.booking_id < b.booking_id

);

```

6.3 Find All Resources with Any Overlap

```

SELECT DISTINCT a.resource

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

```

```
AND a.booking_id <> b.booking_id
```

```
AND a.start_time < b.end_time
```

```
AND a.end_time > b.start_time;
```

Chapter 7 · Pattern 6 — Transitive Closure

Transitive closure answers: 'If $A \rightarrow B$ and $B \rightarrow C$, is A transitively related to C ?' Classic use-cases: social graphs (friend-of-friend), product categories, dependency graphs, access-control inheritance.

Schema — follows (social graph)

```
CREATE TABLE follows (  
  
  follower_id INT,  
  
  followee_id INT,  
  
  PRIMARY KEY (follower_id, followee_id)  
  
);
```

7.1 Friends-of-Friends (2 Hops)

```
SELECT DISTINCT f1.follower_id AS person,  
  
  f2.followee_id AS friend_of_friend  
  
FROM follows AS f1  
  
JOIN follows AS f2 ON f1.followee_id = f2.follower_id  
  
WHERE f2.followee_id <> f1.follower_id -- exclude self  
  
AND NOT EXISTS ( -- exclude direct follows  
  
  SELECT 1 FROM follows AS df  
  
  WHERE df.follower_id = f1.follower_id  
  
  AND df.followee_id = f2.followee_id  
  
);
```

7.2 Full Transitive Closure with Recursive CTE

```
WITH RECURSIVE closure AS (  
  
  SELECT follower_id, followee_id, 1 AS hops  
  
  FROM follows
```

UNION

```
SELECT c.follower_id, f.followee_id, c.hops + 1

FROM closure AS c

JOIN follows AS f ON c.followee_id = f.follower_id

WHERE c.hops < 6 -- depth guard

)

SELECT follower_id, followee_id, MIN(hops) AS shortest_path

FROM closure

GROUP BY follower_id, followee_id;
```

■ *UNION (not UNION ALL) deduplicates automatically preventing infinite loops in cyclic graphs when combined with a depth guard.*

Chapter 8 · Pattern 7 — Time-Series & Gap Detection

Schema — sensor_readings

```
CREATE TABLE sensor_readings (  
  
  sensor_id INT,  
  
  recorded_at TIMESTAMP,  
  
  value DECIMAL(8,2),  
  
  PRIMARY KEY (sensor_id, recorded_at)  
  
);
```

8.1 Detect Gaps > 1 Hour Between Consecutive Readings

```
SELECT a.sensor_id,  
  
  a.recorded_at AS gap_start,  
  
  b.recorded_at AS gap_end,  
  
  EXTRACT(EPOCH FROM (b.recorded_at - a.recorded_at))/3600 AS gap_hours  
  
FROM sensor_readings AS a  
  
JOIN sensor_readings AS b  
  
ON b.sensor_id = a.sensor_id  
  
AND b.recorded_at > a.recorded_at  
  
AND NOT EXISTS (  
  
  SELECT 1 FROM sensor_readings AS mid  
  
  WHERE mid.sensor_id = a.sensor_id  
  
  AND mid.recorded_at > a.recorded_at  
  
  AND mid.recorded_at < b.recorded_at  
  
  ) -- b is the immediately next reading
```

```
WHERE b.recorded_at - a.recorded_at > INTERVAL '1 hour';
```

8.2 Island Detection (Consecutive Active Days)

```
-- Tag each reading with whether the previous day also had a reading

SELECT a.sensor_id,

a.recorded_at::DATE AS day,

CASE WHEN b.sensor_id IS NULL THEN 'START' ELSE 'CONT' END AS island_flag

FROM sensor_readings AS a

LEFT JOIN sensor_readings AS b

ON b.sensor_id = a.sensor_id

AND b.recorded_at::DATE = a.recorded_at::DATE - 1

GROUP BY a.sensor_id, a.recorded_at::DATE, b.sensor_id;
```

8.3 Moving Average (Self Join Approach)

```
-- 3-period moving average per sensor

SELECT a.sensor_id,

a.recorded_at,

a.value,

AVG(b.value) AS moving_avg_3

FROM sensor_readings AS a

JOIN sensor_readings AS b

ON b.sensor_id = a.sensor_id

AND b.recorded_at BETWEEN a.recorded_at - INTERVAL '2 hours'

AND a.recorded_at

GROUP BY a.sensor_id, a.recorded_at, a.value

ORDER BY a.sensor_id, a.recorded_at;
```

Chapter 9 · Pattern 8 — Ranking Without Window Functions

Legacy databases, or specific edge cases, require ranking logic without `ROW_NUMBER()`. Self joins offer a pure-SQL substitute.

9.1 Dense Rank

```
-- Dense rank of employees by salary descending

SELECT a.emp_id,

a.name,

a.salary,

COUNT(DISTINCT b.salary) + 1 AS dense_rank

FROM employees AS a

LEFT JOIN employees AS b ON b.salary > a.salary

GROUP BY a.emp_id, a.name, a.salary

ORDER BY dense_rank;
```

9.2 Row Number (Unique Rank)

```
SELECT a.emp_id,

a.name,

a.salary,

COUNT(b.emp_id) + 1 AS row_number

FROM employees AS a

LEFT JOIN employees AS b

ON b.salary > a.salary

OR (b.salary = a.salary AND b.emp_id < a.emp_id)

GROUP BY a.emp_id, a.name, a.salary

ORDER BY row_number;
```

9.3 Median Salary

```
-- Works in all SQL dialects

SELECT AVG(a.salary) AS median_salary

FROM employees AS a

WHERE (

SELECT COUNT(*) FROM employees AS b WHERE b.salary <= a.salary

) >= (SELECT COUNT(*) / 2.0 FROM employees)

AND (

SELECT COUNT(*) FROM employees AS b WHERE b.salary >= a.salary

) >= (SELECT COUNT(*) / 2.0 FROM employees);
```

Chapter 10 · Performance, Anti-Patterns & Optimisation

10.1 Complexity Classes of Self Join Patterns

Pattern	Complexity	Rows (N=1M)	Recommendation
Hierarchical (1 level)	$O(N)$	1 M	Fast — index manager_id
Within-group pairs	$O(N^2/2)$	500 B	Partition + index dept
Running total	$O(N^2)$	1 T	Use window SUM() OVER
Recursive CTE (depth k)	$O(N \cdot k)$	varies	Depth guard mandatory
Transitive closure	$O(N^2 \cdot k)$	very large	Use graph DB or index
Overlap detection	$O(N^2)$	500 B	Use range index / GiST

10.2 Indexing Checklist

- ✓ JOIN key columns: index manager_id, dept, sensor_id
- ✓ Composite index for equality + range: (sensor_id, recorded_at)
- ✓ For interval overlap: PostgreSQL GiST index on TSRANGE; MySQL SPATIAL on date pairs
- ✓ Covering index: include projected columns to avoid table lookups
- ✓ EXPLAIN / EXPLAIN ANALYZE before and after every optimisation

10.3 Top Anti-Patterns

✗ Missing alias filter (a.id <> b.id)

→ Result doubles or includes same-row comparisons silently.

✗ Self join instead of window function for running totals

→ $O(N^2)$ vs $O(N)$. Kills performance at scale.

✗ Recursive CTE without depth guard

→ Infinite loop on cyclic data. Always add WHERE depth < N.

✗ SELECT * on self join

→ Ambiguous column names; always enumerate with aliases.

✗ Cross join when INNER was intended

→ N^2 rows before filtering. Check your ON clause is present.

10.4 Decision Tree: Self Join vs Alternative

Need to compare a row to adjacent rows?

■ Fixed look-back N rows → LAG/LEAD window function (preferred)

■ Conditional look-back → Self join with correlated subquery

Need to traverse a hierarchy?

■ Fixed known depth (1-3 hops) → Multi-alias self join

■ Unknown/variable depth → Recursive CTE

Need to rank rows?

■ Modern RDBMS → ROW_NUMBER / RANK / DENSE_RANK

■ Legacy / portable → Self join COUNT trick

Need to find overlaps?

■ Temporal intervals → Self join with Allen's overlap predicate

■ Spatial data → GiST / R-Tree spatial index

Chapter 11 · 30 Graded Exercises

Exercises are grouped into three tiers. Work through each before checking the answer key in Chapter 12.

■ BEGINNER (Exercises 1–10)

Ex 1. List every employee alongside their manager's name. Employees with no manager should still appear (show NULL for manager).

Ex 2. Find all employees whose salary is strictly greater than their direct manager's salary.

Ex 3. List all pairs of employees in the same department (each unordered pair once — not A,B and B,A).

Ex 4. Find the highest-earning employee in each department using a self join (no MAX() aggregate, no window function).

Ex 5. List employees who were hired before their direct manager.

Ex 6. Return the name and salary of the 3rd-highest-paid employee company-wide (no LIMIT / TOP, no window function — self join only).

Ex 7. Identify employees who share the same hire_date as at least one other employee.

Ex 8. Find all direct reports of 'Sarah Chen'.

Ex 9. List every employee and the count of peers (same department, different person).

Ex 10. Return all employees who have NO direct reports (i.e., nobody has them as manager).

■ INTERMEDIATE (Exercises 11–20)

Ex 11. For each sale, show the previous sale amount by the same employee (NULL if it is their first sale). Use only a self join — no window functions.

Ex 12. Find all bookings that overlap with at least one other booking for the same resource. Return booking_id, resource, and the count of overlapping bookings.

Ex 13. Compute the running total of salary for employees sorted by hire_date using a self join.

Ex 14. Find all employees who have exactly 2 peers earning more than them in their department.

Ex 15. List all resource names that have at least one pair of non-overlapping bookings that together cover an uninterrupted span of > 8 hours.

Ex 16. Write a recursive CTE to display the full reporting chain from any leaf employee up to the root, as a single concatenated path string.

Ex 17. Find the average salary of each employee's peers (same department, excluding self). Show employees earning above their peer average.

Ex 18. Using the sensor_readings table, find all sensors that had a gap of more than 2 hours between any two consecutive readings.

Ex 19. Detect duplicate bookings (same resource, same start_time, same end_time) and return the booking_id to delete (keep the lowest booking_id).

Ex 20. Using a self join compute the dense rank of each employee by salary descending, partitioned by department.

■ ADVANCED (Exercises 21–30)

Ex 21. Build the full transitive closure of the employee hierarchy: for every (ancestor, descendant) pair show the shortest hop count.

Ex 22. Identify 'salary inversion chains': find cases where A manages B, B manages C, but A earns less than C (two-level salary anomaly).

Ex 23. For each employee, find the co-workers (same dept) hired within 90 days of them (before or after), forming 'hiring cohort pairs'.

Ex 24. Given the follows table, find all users who are mutual followers (A follows B AND B follows A).

Ex 25. Detect all sensor_ids where readings are monotonically increasing for at least 5 consecutive timestamps (no drop in value).

Ex 26. Using only self joins and aggregates (no window functions), compute a 3-point centred moving average of sensor values.

Ex 27. Find all employees at depth exactly 2 from the root of their reporting tree (i.e., two levels below a root), returning their names and the root's name.

Ex 28. Write a query that returns, for each booking, the total 'busy time' occupied by all overlapping bookings including itself, for the same resource.

Ex 29. Find the 'most isolated' employee: the one whose nearest colleague (by hire_date distance) is farthest away in time.

Ex 30. CHALLENGE — Write a single SQL statement (CTEs allowed) that produces a complete org-chart report: emp_id, name, depth, path, peer_count, salary_rank_in_dept, and a flag 'OVERPAID' if their salary exceeds their manager's by more than 20%.

Chapter 12 · Full Answer Key

Answers 1–10 (Beginner)

Answer 1

```
SELECT e.name AS employee, m.name AS manager

FROM employees AS e

LEFT JOIN employees AS m ON e.manager_id = m.emp_id;
```

Answer 2

```
SELECT e.name AS employee, e.salary,

m.name AS manager, m.salary AS manager_salary

FROM employees AS e

JOIN employees AS m ON e.manager_id = m.emp_id

WHERE e.salary > m.salary;
```

Answer 3

```
SELECT a.name AS emp1, b.name AS emp2, a.dept

FROM employees AS a

JOIN employees AS b

ON a.dept = b.dept

AND a.emp_id < b.emp_id;
```

Answer 4

```
SELECT a.dept, a.name, a.salary

FROM employees AS a

WHERE NOT EXISTS (

SELECT 1 FROM employees AS b

WHERE b.dept = a.dept
```

```
AND b.salary > a.salary

);
```

Answer 5

```
SELECT e.name AS employee, e.hire_date,

m.name AS manager, m.hire_date AS mgr_hire

FROM employees AS e

JOIN employees AS m ON e.manager_id = m.emp_id

WHERE e.hire_date < m.hire_date;
```

Answer 6

```
-- Exactly 2 employees have a higher salary than the 3rd-highest

SELECT name, salary

FROM employees AS a

WHERE 2 = (

SELECT COUNT(DISTINCT b.salary)

FROM employees AS b

WHERE b.salary > a.salary

);
```

Answer 7

```
SELECT DISTINCT a.name, a.hire_date

FROM employees AS a

JOIN employees AS b

ON a.hire_date = b.hire_date

AND a.emp_id <> b.emp_id;
```

Answer 8

```
SELECT e.name AS direct_report

FROM employees AS e

JOIN employees AS m ON e.manager_id = m.emp_id

WHERE m.name = 'Sarah Chen';
```

Answer 9

```
SELECT a.name, a.dept, COUNT(b.emp_id) AS peer_count

FROM employees AS a

LEFT JOIN employees AS b

ON a.dept = b.dept

AND a.emp_id <> b.emp_id

GROUP BY a.emp_id, a.name, a.dept;
```

Answer 10

```
SELECT e.name

FROM employees AS e

LEFT JOIN employees AS sub ON sub.manager_id = e.emp_id

WHERE sub.emp_id IS NULL;
```

Answers 11–20 (Intermediate)

Answer 11

```
SELECT curr.emp_id, curr.sale_date, curr.amount,

prev.sale_date AS prev_date, prev.amount AS prev_amount

FROM sales AS curr

LEFT JOIN sales AS prev

ON prev.emp_id = curr.emp_id

AND prev.sale_date = (
```

```
SELECT MAX(p.sale_date)

FROM sales AS p

WHERE p.emp_id = curr.emp_id

AND p.sale_date < curr.sale_date

);
```

Answer 12

```
SELECT a.booking_id, a.resource,

COUNT(b.booking_id) AS overlap_count

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.booking_id <> b.booking_id

AND a.start_time < b.end_time

AND a.end_time > b.start_time

GROUP BY a.booking_id, a.resource;
```

Answer 13

```
SELECT a.emp_id, a.name, a.hire_date, a.salary,

SUM(b.salary) AS running_salary_total

FROM employees AS a

JOIN employees AS b ON b.hire_date <= a.hire_date

GROUP BY a.emp_id, a.name, a.hire_date, a.salary

ORDER BY a.hire_date;
```

Answer 14

```
SELECT a.name, a.dept, a.salary
```

```

FROM employees AS a

WHERE 2 = (

SELECT COUNT(*)

FROM employees AS b

WHERE b.dept = a.dept

AND b.salary > a.salary

);

```

Answer 15

```

-- Pairs of same-resource bookings that don't overlap and span > 8h together

SELECT DISTINCT a.resource

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.booking_id < b.booking_id

AND a.end_time <= b.start_time -- non-overlapping

AND EXTRACT(EPOCH FROM (

b.end_time - a.start_time)) / 3600 > 8;

```

Answer 16

```

WITH RECURSIVE chain AS (

SELECT emp_id, name, manager_id,

CAST(name AS VARCHAR(1000)) AS path

FROM employees

WHERE manager_id IS NULL

UNION ALL

```

```

SELECT e.emp_id, e.name, e.manager_id,

CONCAT(c.path, ' → ', e.name)

FROM employees AS e

JOIN chain AS c ON e.manager_id = c.emp_id

)

SELECT path FROM chain ORDER BY path;

```

Answer 17

```

SELECT a.name, a.dept, a.salary,

AVG(b.salary) AS peer_avg

FROM employees AS a

JOIN employees AS b

ON a.dept = b.dept

AND a.emp_id <> b.emp_id

GROUP BY a.emp_id, a.name, a.dept, a.salary

HAVING a.salary > AVG(b.salary);

```

Answer 18

```

SELECT DISTINCT a.sensor_id

FROM sensor_readings AS a

JOIN sensor_readings AS b

ON b.sensor_id = a.sensor_id

AND b.recorded_at > a.recorded_at

AND NOT EXISTS (

SELECT 1 FROM sensor_readings AS m

WHERE m.sensor_id = a.sensor_id

```

```
AND m.recorded_at > a.recorded_at

AND m.recorded_at < b.recorded_at

)

WHERE b.recorded_at - a.recorded_at > INTERVAL '2 hours';
```

Answer 19

```
DELETE FROM bookings

WHERE booking_id IN (

SELECT b.booking_id

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.start_time = b.start_time

AND a.end_time = b.end_time

AND a.booking_id < b.booking_id

);
```

Answer 20

```
SELECT a.emp_id, a.name, a.dept, a.salary,

COUNT(DISTINCT b.salary) + 1 AS dense_rank_in_dept

FROM employees AS a

LEFT JOIN employees AS b

ON b.dept = a.dept

AND b.salary > a.salary

GROUP BY a.emp_id, a.name, a.dept, a.salary

ORDER BY a.dept, dense_rank_in_dept;
```

Answers 21–30 (Advanced)

Answer 21

```
WITH RECURSIVE closure AS (

  SELECT emp_id AS ancestor, emp_id AS descendant, 0 AS hops

  FROM employees

  UNION

  SELECT c.ancestor, e.emp_id, c.hops + 1

  FROM closure AS c

  JOIN employees AS e ON e.manager_id = c.descendant

  WHERE c.hops < 10

)

SELECT ancestor, descendant, MIN(hops) AS shortest_hops

FROM closure WHERE ancestor <> descendant

GROUP BY ancestor, descendant;
```

Answer 22

```
-- A manages B, B manages C, but salary(A) < salary(C)

SELECT a.name AS top_mgr, a.salary AS top_sal,

b.name AS mid_mgr, b.salary AS mid_sal,

c.name AS subordinate, c.salary AS sub_sal

FROM employees AS a -- grandparent

JOIN employees AS b ON b.manager_id = a.emp_id -- child

JOIN employees AS c ON c.manager_id = b.emp_id -- grandchild

WHERE a.salary < c.salary;
```

Answer 23

```
SELECT a.name AS emp1, b.name AS emp2,
```

```

a.hire_date, b.hire_date AS peer_hire,

ABS(a.hire_date - b.hire_date) AS days_apart

FROM employees AS a

JOIN employees AS b

ON a.dept = b.dept

AND a.emp_id < b.emp_id

AND ABS(EXTRACT(DAY FROM (a.hire_date - b.hire_date))) <= 90

ORDER BY a.name;

```

Answer 24

```

SELECT a.follower_id AS user1, a.followee_id AS user2

FROM follows AS a

JOIN follows AS b

ON a.follower_id = b.followee_id

AND a.followee_id = b.follower_id

AND a.follower_id < a.followee_id; -- unique pairs

```

Answer 25

```

-- 5 consecutive increasing readings: r1 < r2 < r3 < r4 < r5

SELECT DISTINCT r1.sensor_id

FROM sensor_readings AS r1

JOIN sensor_readings AS r2 ON r2.sensor_id=r1.sensor_id AND r2.recorded_at >
r1.recorded_at

JOIN sensor_readings AS r3 ON r3.sensor_id=r1.sensor_id AND r3.recorded_at >
r2.recorded_at

JOIN sensor_readings AS r4 ON r4.sensor_id=r1.sensor_id AND r4.recorded_at >
r3.recorded_at

```

```

JOIN sensor_readings AS r5 ON r5.sensor_id=r1.sensor_id AND r5.recorded_at >
r4.recorded_at

WHERE r1.value < r2.value

AND r2.value < r3.value

AND r3.value < r4.value

AND r4.value < r5.value

AND NOT EXISTS ( -- consecutive gap check for r1→r2

SELECT 1 FROM sensor_readings x

WHERE x.sensor_id=r1.sensor_id

AND x.recorded_at > r1.recorded_at AND x.recorded_at < r2.recorded_at);

```

Answer 26

```

-- 3-point centred moving average: prev, curr, next

SELECT curr.sensor_id, curr.recorded_at, curr.value,

(prev.value + curr.value + next_r.value) / 3.0 AS moving_avg

FROM sensor_readings AS curr

JOIN sensor_readings AS prev

ON prev.sensor_id = curr.sensor_id

AND prev.recorded_at = (

SELECT MAX(p.recorded_at) FROM sensor_readings p

WHERE p.sensor_id = curr.sensor_id

AND p.recorded_at < curr.recorded_at)

JOIN sensor_readings AS next_r

ON next_r.sensor_id = curr.sensor_id

AND next_r.recorded_at = (

SELECT MIN(n.recorded_at) FROM sensor_readings n

```

```
WHERE n.sensor_id = curr.sensor_id

AND n.recorded_at > curr.recorded_at);
```

Answer 27

```
-- Depth-2 nodes: grandchildren of roots

SELECT gc.name AS grandchild, root.name AS root_name

FROM employees AS root

JOIN employees AS child ON child.manager_id = root.emp_id

JOIN employees AS gc ON gc.manager_id = child.emp_id

WHERE root.manager_id IS NULL;
```

Answer 28

```
SELECT a.booking_id, a.resource,

SUM(EXTRACT(EPOCH FROM (

LEAST(a.end_time,b.end_time) -

GREATEST(a.start_time,b.start_time)

))) / 3600.0 AS total_busy_hours

FROM bookings AS a

JOIN bookings AS b

ON a.resource = b.resource

AND a.start_time < b.end_time

AND a.end_time > b.start_time

GROUP BY a.booking_id, a.resource;
```

Answer 29

```
-- For each employee find the closest colleague by hire_date (excl. self)

-- Then return the one with the largest minimum distance
```

```

SELECT a.name, MIN(ABS(a.hire_date - b.hire_date)) AS nearest_days

FROM employees AS a

JOIN employees AS b ON a.emp_id <> b.emp_id

GROUP BY a.emp_id, a.name

ORDER BY nearest_days DESC

LIMIT 1;

```

Answer 30

```

-- CHALLENGE: Full org-chart report

WITH RECURSIVE tree AS (

SELECT emp_id, name, manager_id, salary, dept,

0 AS depth,

CAST(name AS VARCHAR(2000)) AS path

FROM employees WHERE manager_id IS NULL

UNION ALL

SELECT e.emp_id, e.name, e.manager_id, e.salary, e.dept,

t.depth + 1,

CONCAT(t.path, ' → ', e.name)

FROM employees e JOIN tree t ON e.manager_id = t.emp_id

),

ranked AS (

SELECT tr.*,

COUNT(b.emp_id) AS peer_count,

COUNT(DISTINCT hi.salary) + 1 AS salary_rank_in_dept

FROM tree tr

```

```
LEFT JOIN employees b

ON b.dept = tr.dept AND b.emp_id <> tr.emp_id

LEFT JOIN employees hi

ON hi.dept = tr.dept AND hi.salary > tr.salary

GROUP BY tr.emp_id, tr.name, tr.manager_id, tr.salary,

tr.dept, tr.depth, tr.path

)

SELECT r.emp_id, r.name, r.depth, r.path,

r.peer_count, r.salary_rank_in_dept,

CASE WHEN r.salary > m.salary * 1.20

THEN 'OVERPAID' ELSE 'OK' END AS flag

FROM ranked r

LEFT JOIN employees m ON r.manager_id = m.emp_id

ORDER BY r.depth, r.dept, r.salary_rank_in_dept;
```

— End of Guide —

Master every pattern, revisit the exercises after a week without looking at answers, and profile your queries on real datasets. That is the senior developer standard.